

Advanced Structural and Tensor Analysis

Block 2 — Modern Workflow in Geology

Ondrej Lexa

Institute of Petrology and Structural Geology (IPSG)
Charles University, Prague, Czechia

8 October 2026

This Block (11:15–12:30)

Objective

Functions that would take hours by hand: kinematic analysis of faults, finite strain and progressive deformation, stress, and stress inversion.

1. Fault-slip data: Angelier and Hoeppner plots, P/T axes, right dihedral
2. Deformation gradient, strain ellipsoid, progressive deformation
3. Superposition of deformations
4. Strain analysis from markers: R_f/ϕ , Robin mean ellipse/ellipsoid
5. Stress tensor and stress inversion

Notebook: `notebooks/02_apsg_advanced.ipynb`. Tensors combine with @ — never *.

Kinematics and Faults

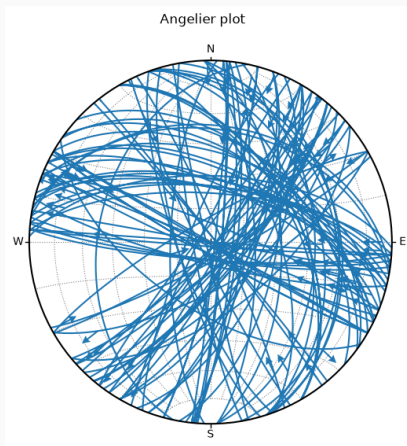
Fault-Slip Data

```
faults = faultset.from_csv("data/mele.csv")  
# columns: fazi, finc, lazi, linc, sense  
# or: pd.read_csv("data/mele.csv").fault()
```

```
s = StereoNet(title="Angelier plot")  
s.fault(faults)  
s.show()
```

```
s = StereoNet(title="Hoeppner plot")  
s.hoeppner(faults)
```

Sense: 1/"n" normal, -1/"r" reverse, "s" sinistral, "d" dextral.



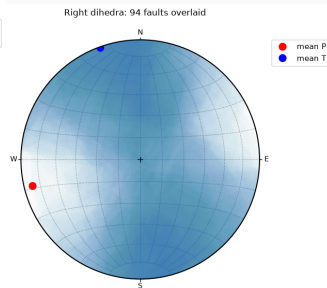
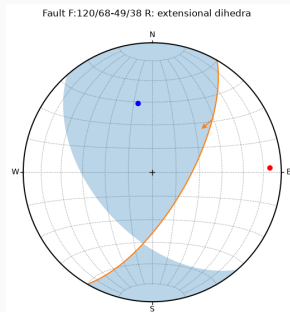
94 faults, Angelier plot

Right Dihedra

```
f = faults[62] # F:120/68-49/38 R
s = StereoNet()
s.dihedra(f) # extensional dihedra
s.fault(f)
s.line(f.p, color="r", label="P")
s.line(f.t, color="b", label="T")

s = StereoNet()
s.dihedra(faults) # all 94 faults
```

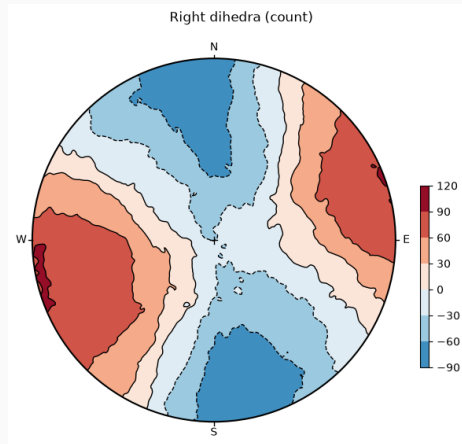
Fault plane + auxiliary plane split the sphere into two compressional and two extensional dihedra. Where many faults overlap, the shading is darkest (σ_3); unshaded areas are the σ_1 candidates.



P/T Axes and Right-Dihedra Count

```
P, T = faults.p, faults.t # LineationSets  
P.ortensor().eigenlins(0) # L:256/8  
  
g = StereoGrid()  
g.angmech(faults) # Angelier-Mechler  
s = StereoNet()  
s.contour(g, clip=False, colorbar=True)  
print(g) # max 90 at L:268/2, min -84 at L  
         # :167/11
```

Model-free first look at the kinematics: every direction is counted as compressional (+) or extensional (−) for each fault.



Finite Strain

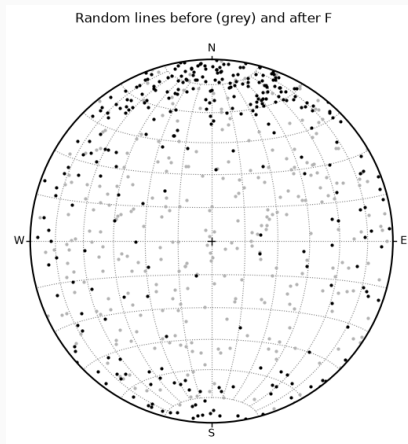
Deformation Gradient

```
F = defgrad.from_comp(xx=2, zz=0.5, xz=1)
F @ vec("x") # deformed vector
F.R, F.U # rotation, stretch
defgrad.from_ratios(Rxy=2, Ryz=3)

Lr = linset.random_fisher(position=lin(0, 90),
                           kappa=1, n=300)

s = StereoNet()
s.line(Lr, color="0.7", ms=2)
s.line(Lr.transform(F), color="k", ms=2)
```

Any feature or set can be `.transform(F)`-ed —
planes are handled correctly through their poles.



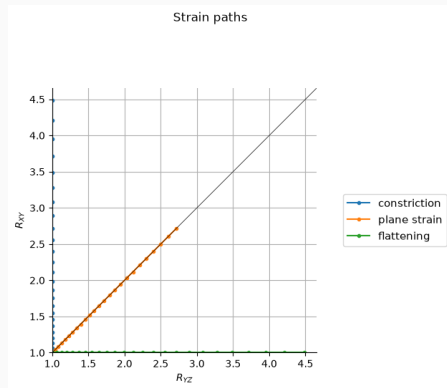

```
B = ellipsoid.from_defgrad(F) # Finger tensor  $B = F F^T$   
B.Rxy, B.Ryz, B.k, B.lode # 2.25, 2.25, 1.00, 0.00  
B.eigenlins() # X, Y, Z directions: L:0/6, L:270/0, L:180/84
```

- Shape descriptors: Rxy, Ryz, Flinn k, Lode lode, P_j, T, ...
- Principal directions: B.eigenlins() (X, Y, Z), B.eigenfols() (principal planes)
- Fabric plots: FlinnPlot, RamsayPlot, VollmerPlot, HsuPlot — .point(E) or .path(Eset)
- The same plots take orientation tensors (ortensor) of measured fabrics

Progressive Deformation

```
Lss = velgrad.from_comp(xz=1) # simple shear
Lss.kinematic_vorticity #  $Wk = 1.0$ 
Lss.defgrad(time=2) #  $F = \exp(L t)$ 

flows = {
    "constriction": velgrad.from_comp(
        xx=1, yy=-0.5, zz=-0.5),
    "plane strain": velgrad.from_comp(xx=1, zz=-1),
    "flattening": velgrad.from_comp(
        xx=0.5, yy=0.5, zz=-1)}
fp = FlinnPlot()
for name, L in flows.items():
    E = ellipsoidset([ellipsoid.from_defgrad(
        L.defgrad(time=t)) for t in times])
    fp.path(E, label=name, marker=".")
```

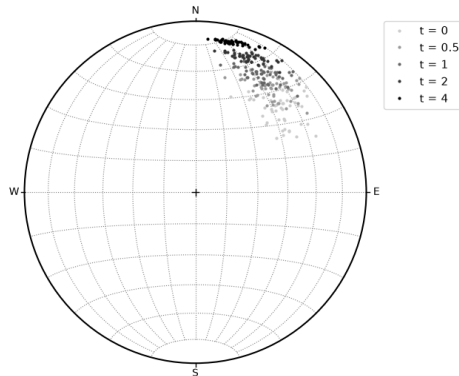


Rotating Fabrics during Flow

```
L0 = linset.random_fisher(position=lin(45, 30),  
                           kappa=50, n=50)  
s = StereoNet()  
for t in [0, 0.5, 1, 2, 4]:  
    s.line(L0.transform(Lss.defgrad(time=t)),  
          ms=2, label=f"t = {t}")  
s.show()
```

Lineations progressively rotate towards the shear direction ($x = \text{north}$) — the basis for interpreting stretching lineations in shear zones.

Lineations during simple shear ($x = 0/0$, shear plane = horizontal)



Superposition of Deformations

```
F1 = defgrad.from_comp(xx=2, zz=0.5) # D1: pure shear
F2 = rotation.from_axisangle(lin(0, 90), 60) # rigid rotation about vertical
F3 = defgrad.from_comp(xy=1) # D2: simple shear

ellipsoid.from_defgrad(F3 @ F2 @ F1) # D1 -> rot -> D2: Rxy=5.23 Ryz=1.24
ellipsoid.from_defgrad(F1 @ F2 @ F3) # D2 -> rot -> D1: Rxy=1.32 Ryz=2.47
```

Order matters

Finite deformations combine by matrix multiplication with the **later event on the left**: $\mathbf{F} = \mathbf{F}_2 \cdot \mathbf{F}_1$. Matrix products do not commute — the same two events in a different order give a different finite strain.

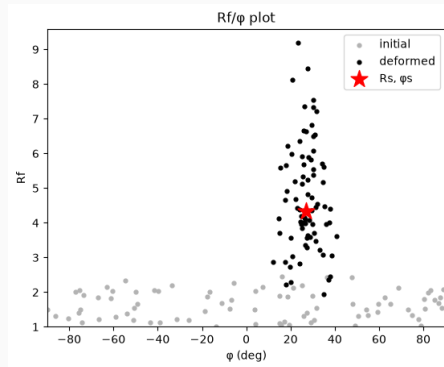
Strain Analysis

Strain from Markers: R_f/ϕ

```
pebbles0 = ellipset([ # 80 random markers
    ellipse.from_ratio(R=r)
        .transform(rotation2.from_angle(p))
    for r, p in zip(Ri, phi_i)]) # Ri 1-2.5
F = defgrad2.from_comp(xy=1.5) # simple shear
pebbles = pebbles0.transform(F)

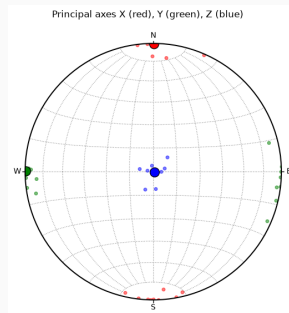
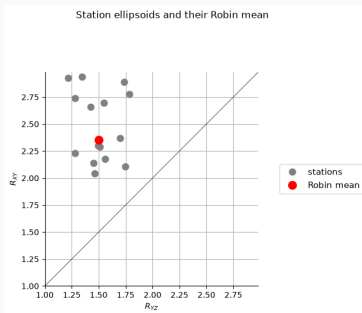
ellipse.from_defgrad(F) # true: ar 4, ori 26.6
pebbles.rfphi() # Rs = 4.34, phi = 27.0
pebbles.robin() # Rs = 4.34, phi = 27.0
pebbles.ar.mean() # 4.72 - biased
```

`rfphi()`: vector mean of $\ln R_f$ at 2ϕ . `robin()`:
log-matrix mean (Robin 1977) — identical in 2D.



Mean Strain Ellipsoid (Robin 1977)

```
stations = ellipsoidset([...]) # 15 strain ellipsoids, axes scattered by ~15 deg
Emean = stations.robin() # Rxy=2.35 Ryz=1.50 k=2.70 volume=1.00
ellipsoid(np.mean([np.asarray(E) for E in stations], axis=0)) # volume=1.10
fp = FlinnPlot()
fp.point(stations, color="0.5"); fp.point(Emean, color="r", ms=10)
s = StereoNet()
s.tensor(stations, planes=False, ms=4) # principal axes of all stations
```



Stress

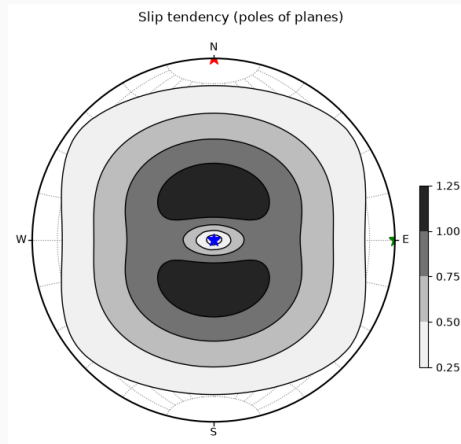
Stress Tensor

```
S = stress.from_comp(xx=8, yy=5, zz=1)
S.sigma1, S.sigma2, S.sigma3 # 8, 5, 1
lin(S.sigma1dir) # L:0/0
S.shape_ratio # 0.43

n = fol(120, 60)
S.normal_stress(n) # 4.56
S.shear_stress(n) # 2.34
S.slip_tendency(n) # 0.51
S.fault(n) # predicted slip (Wallace-Bott)

g = StereoGrid()
g.apply_func(S.slip_tendency)
StereoNet().contour(g, colorbar=True)
```

Geological convention: **compression positive.**



Stress Inversion — Test First

```
synthetic = faultset([S.fault(n) for n in vecset.gss(n=200)])  
Sinv = synthetic.stress_inversion()  
# true sigma1: L:0/0 R = 0.43  
# inverted sigma1: L:0/0 R = 0.41
```

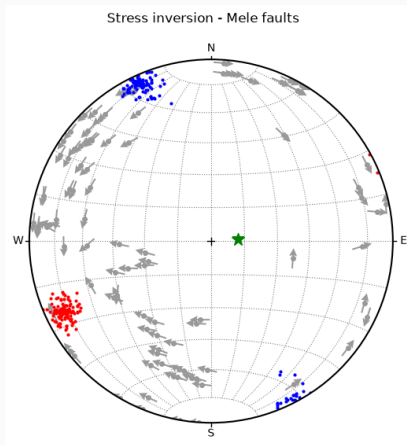
- `faultset.stress_inversion()` — linear least-squares method of Michael (1984): the reduced stress tensor that best explains all striations at once
- Faults generated from a *known* tensor must return it — a habit worth keeping for any black-box method

Stress Inversion — Real Data

```
Smele = faults.stress_inversion()
lin(Smele.sigma1dir) # L:244/11
lin(Smele.sigma3dir) # L:335/4
Smele.shape_ratio # 0.59

misfit = faults.angular_misfit(Smele)
np.median(misfit) # 22.9 deg

boot = faults.stress_inversion(
    bootstrap=True, n=100)
s = StereoNet()
s.hoeppner(faults, color="0.6")
s.line(G([lin(b.sigma1dir) for b in boot]), color="
    r")
s.stress(Smele)
```



σ_1 (red), σ_3 (blue): bootstrap clouds

Hands-on

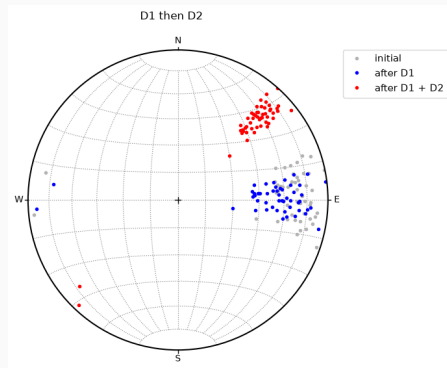
Hands-on Exercise

A. Progressive deformation and superposition

1. 50 lineations around `lin(90, 20)`
2. D1: pure shear `velgrad.from_comp(xx=-1, zz=1, time=0.5`; D2: simple shear `from_comp(xy=1, time=1`
3. Plot initial / D1 / D1+D2; ellipsoid of the total — does swapping the order matter?

B. Brittle structures

4. Keep Mele faults with $\text{misfit} < 30^\circ$ and invert again
5. Save Angelier plot + principal stresses as PDF



Part A result (solution in the notebook)

Recap

Task	Code
Fault data & plots	<code>faultset.from_csv, s.fault, s.hoeppner</code>
Kinematics	<code>fs.p, fs.t, s.dihedra(fs), StereoGrid().angmech(fs)</code>
Finite strain	<code>defgrad, F.R, F.U, ellipsoid.from_defgrad(F)</code>
Flow	<code>velgrad, L.defgrad(time=t), L.kinematic_vorticity</code>
Superposition	<code>F2 @ F1 (later event on the left)</code>
Strain markers	<code>ellipseset.rfphi(), .robin(), ellipsoidset.robin()</code>
Stress	<code>stress.from_comp, .sigma1dir, .shear_stress(n), .fault(n)</code>
Inversion	<code>fs.stress_inversion(), fs.angular_misfit(S), bootstrap=True</code>

After lunch: petrological data with petropandas